

Chapter 11

인덱스 튜닝

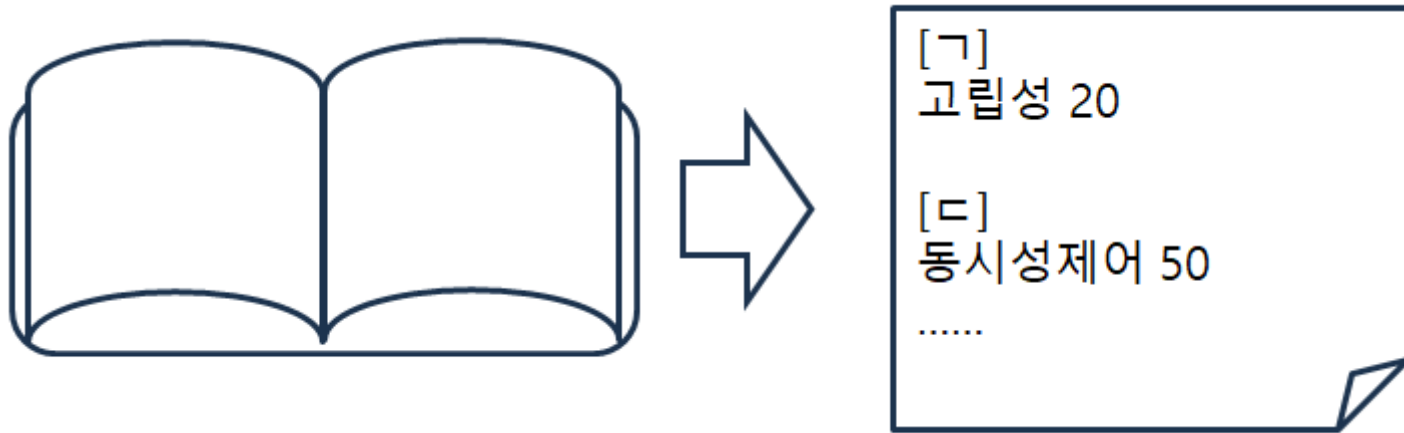
Outline

- 11.1 인덱스 기본 원리
- 11.2 테이블 액세스 최소화
- 11.3 인덱스 스캔 효율화
- 11.4 인덱스 설계

11.1 인덱스 기본 원리

인덱스의 개념

데이터베이스에서 데이터의 검색 속도를 향상시키기 위해 사용되는 데이터 구조



책 맨 뒤에 ‘색인’ 페이지가 있는데 이 페이지로 원하는 정보를 효율적으로 찾을 수 있다.

11.1 인덱스 기본 원리

인덱스 구조

인덱스는 테이블을 효율적으로 조회하기 위해 특정열로 정렬된 데이터 구조를 가짐

사원 테이블

사원번호	사원명	부서번호	연락처
10011	스미스	10	010-1123-3214
10014	존스	30	010-1123-8972
10013	마이클	20	010-1123-7461
10012	제인	10	010-1123-4212

사원 인덱스

사원번호	부서번호
10011	10
10012	10
10013	20
10014	30

정렬된 인덱스 자료구조를 먼저 탐색을 하게 되면 빠른 속도로 검색이 가능하다.

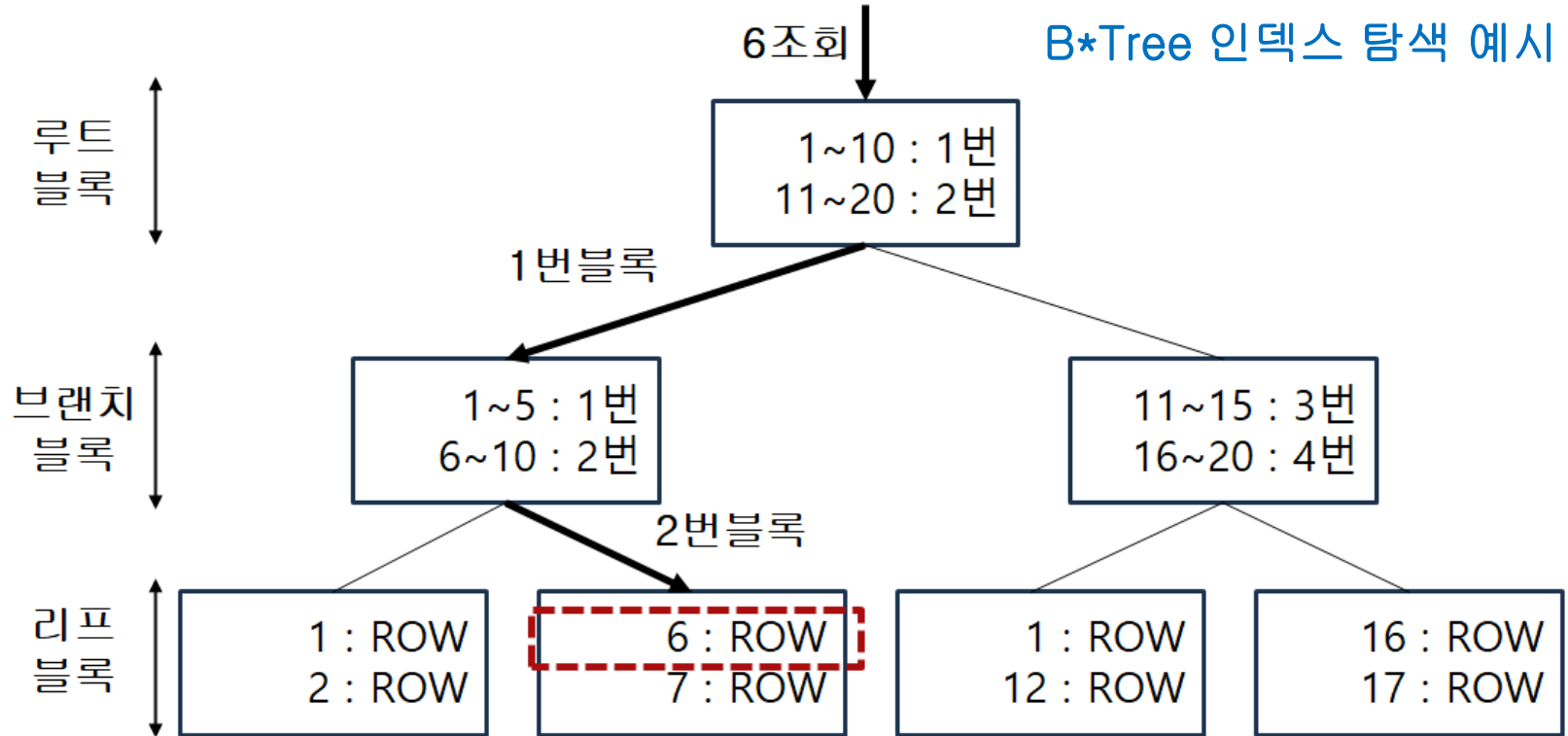
11.1 인덱스 기본 원리

인덱스에 NULL 값 저장

구분	Oracle	SQL Server
NULL 값 저장	구성 컬럼 중 하나라도 NULL이 아니면 저장	구성 컬럼이 모두 NULL이라도 저장
저장 위치	맨 뒤에 저장	맨 앞에 저장

11.1 인덱스 기본 원리

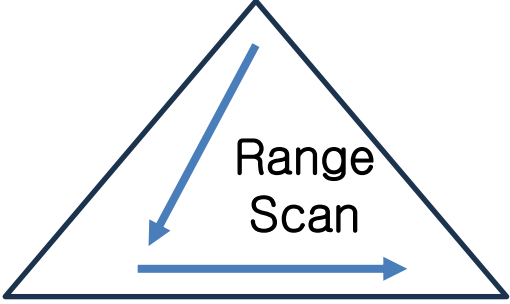
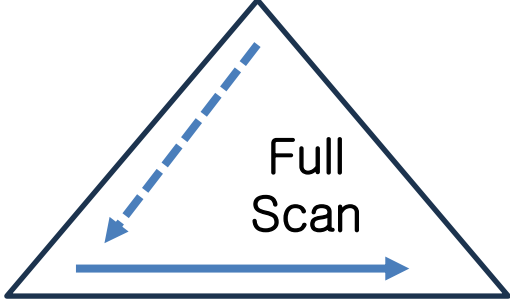
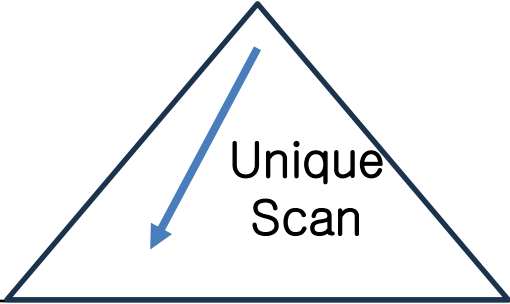
인덱스 탐색



- ① 6의 값을 조회하려면 루트(Root)블록에서 6의 값이 포함된 1번 블록 이동
- ② 브랜치(Branch) 블록에서 6의 값이 포함된 2번 블록 이동
- ③ 리프(Leaf) 블록에서 6값의 ROW정보를 탐색 완료

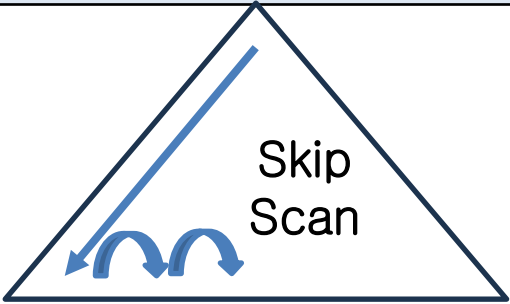
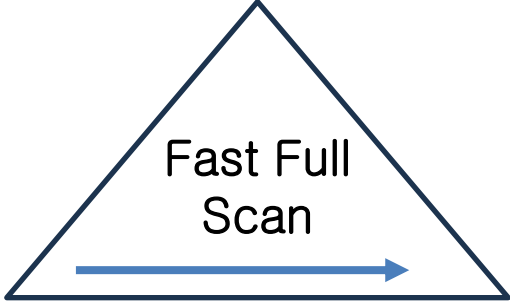
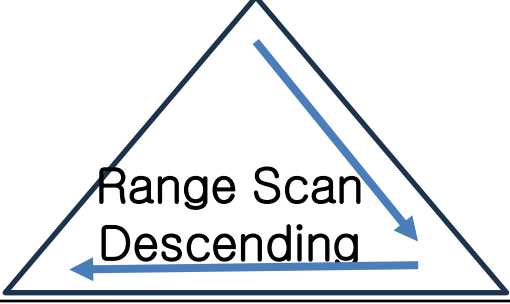
11.1 인덱스 기본 원리

인덱스 스캔 방식

스캔방식	동작방식	설명
Range Scan		- 루트 노드에서 리프 노드까지 수직적 탐색하고, 필요한 범위 만큼 수평적 스캔하는 방식
Full Scan		- 수직적 탐색 없이 인덱스 리프 블록을 전체를 수평적으로 탐색하는 방식
Unique Scan		- Unique Index 를 사용하여 단 하나의 데이터를 추출하는 방식

11.1 인덱스 기본 원리

인덱스 스캔 방식

스캔방식	동작방식	설명
Skip Scan	 Skip Scan	- 멀티 컬럼 인덱스에서 후행 컬럼을 조건절에 사용시 활용
Fast Full Scan	 Fast Full Scan	- 인덱스 트리 구조를 무시하고 인덱스 세그먼트 전체를 Multiblock Read 방식으로 스캔
Range Scan Descending	 Range Scan Descending	- Index Range Scan 과 동일하나 뒤에서부터 앞으로 내림차순으로 스캔해 내림차순 결과를 반환

11.1 인덱스 기본 원리

Index Range Scan의 실행계획 제어

```
EXPLAIN PLAN SET STATEMENT_ID = 'PLAN1' INTO PLAN_TABLE FOR  
SELECT deptno FROM emp WHERE deptno = 30;
```

```
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY(null,null,'ALL'));
```

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		4	12	1 (0)	00:00:01
* 1	INDEX RANGE SCAN	IDX_EMP_05	4	12	1 (0)	00:00:01

- 선두컬럼을 가공하지 않은 상태로 조회에 필요한 범위만 스캔

11.1 인덱스 기본 원리

Index Full Scan의 실행계획 제어

```
CREATE INDEX IDX_EMP_06 ON emp(ename, sal);
```

```
EXPLAIN PLAN SET STATEMENT_ID = 'PLAN1' INTO PLAN_TABLE FOR  
SELECT * FROM emp WHERE sal > 1000 ORDER BY ename;
```

```
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY(null,null,'ALL'));
```

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		6	234	2 (0)	00:00:01
1	TABLE ACCESS BY INDEX ROWID	EMP	6	234	2 (0)	00:00:01
* 2	INDEX FULL SCAN	IDX_EMP_06	6		1 (0)	00:00:01

- 선두컬럼(ename)이 조건절에 없으면 Table Full Scan으로 실행 계획을 고려한다.
- 테이블 용량이 크다면 Table Full Scan보다는 Index Full Scan으로 다시 실행계획을 고려한다.
- Single Block Read 방식

11.1 인덱스 기본 원리

Index Unique Scan의 실행계획 제어

```
EXPLAIN PLAN SET STATEMENT_ID = 'PLAN1' INTO PLAN_TABLE FOR
SELECT * FROM emp WHERE empno = 7369;

SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY(null,null,'ALL'));
```

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		1	39	1 (0)	00:00:01
1	TABLE ACCESS BY INDEX ROWID	EMP	1	39	1 (0)	00:00:01
* 2	INDEX UNIQUE SCAN	PK_EMP	1		0 (0)	00:00:01

- 인덱스 키 컬럼을 모두 등치(=) 조건으로 조회할 경우
- 키 컬럼이 모두 등치이기 때문에 한 건만 찾고 더 이상 탐색하지 않음

11.1 인덱스 기본 원리

Index Skip Scan의 실행계획 제어

```
EXPLAIN PLAN SET STATEMENT_ID = 'PLAN1' INTO PLAN_TABLE FOR
SELECT /*+ index_ss(emp idx_emp_06) */ FROM emp
WHERE sal BETWEEN 1000 AND 4000;

SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY(null,null,'ALL'));
```

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		7	273	2 (0)	00:00:01
1	TABLE ACCESS BY INDEX ROWID BATCHED	EMP	7	273	2 (0)	00:00:01
* 2	INDEX SKIP SCAN	IDX_EMP_06	7		1 (0)	00:00:01

- 인덱스 선두 컬럼을 뺀 상태에서 포함할 가능성이 있는 리프 블록을 골라서 스캔하는 방식
- 선두 컬럼의 Distinct Value 개수가 적고, 후행 컬럼이 Distinct Value 개수가 많을 때 후행 컬럼으로 조회 시 작동
- `index_ss` : Index Skip Scan을 유도하는 힌트

11.1 인덱스 기본 원리

Index Fast Full Scan의 실행계획 제어

```
CREATE TABLE FFS AS SELECT * FROM EMP;
```

```
INSERT INTO FFS SELECT * FROM EMP;
```

```
CREATE INDEX IDX_FFS_01 ON FFS(EMPNO);
```

- 대용량 테이블을 만들어서 Index Fast Full Scan 실행계획 확인
- **INSERT INTO FFS SELECT * FROM EMP;** 반복 실행
- 600 Row 이상 되도록 반복 저장

11.1 인덱스 기본 원리

Index Fast Full Scan의 실행계획 제어

```
EXPLAIN PLAN SET STATEMENT_ID = 'PLAN1' INTO PLAN_TABLE FOR  
SELECT COUNT(EMPNO) FROM FFS;  
  
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY(null,null,'ALL'));
```

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		1	13	3 (0)	00:00:01
1	SORT AGGREGATE		1	13		
2	INDEX FAST FULL SCAN	IDX_FFS_01	651	8463	3 (0)	00:00:01

- 세그먼트 전체를 스캔하고 Full Scan이기 때문에 순서 보장 안됨
- 병렬 스캔 가능
- Multi Block Read 방식

11.1 인덱스 기본 원리

Index Range Scan Descending의 실행계획 제어

```
EXPLAIN PLAN SET STATEMENT_ID = 'PLAN1' INTO PLAN_TABLE FOR
SELECT * FROM emp WHERE empno > 0
ORDER BY empno desc;
```

```
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY(null,null,'ALL'));
```

Id	Operation	Name	Rows	Bytes	Cost	(%CPU)	Time
0	SELECT STATEMENT		7	273	2	(0)	00:00:01
1	TABLE ACCESS BY INDEX ROWID	EMP	7	273	2	(0)	00:00:01
* 2	INDEX RANGE SCAN DESCENDING	PK_EMP	7		1	(0)	00:00:01

- 내림차순 정렬할때 내림차순 컬럼이 index가 있으면
내림차순으로 index를 읽는 실행계획을 수립
- MAX 값을 읽을 때도 이 실행계획으로 수립
- `index_desc` 힌트로 유도 가능

11.1 인덱스 기본 원리

인덱스 종류

종류	설명	단점
B*Tree 인덱스	- 인덱스 루트에서 리프 블록까지 어떤 값으로 탐색하더라도 읽는 블록 수가 같고 루트로부터 리프 블록까지의 높이 (height)가 모두 동일	- Unbalanced Index - Index Skew - Index Sparse
비트맵 인덱스	- 0과 1의 값을 가지는 비트맵 자료구조를 사용하는 인덱스	- 레코드 하나만 변경되더라도 비트맵 범위 모든 레코드에 Lock이 걸려 DML 부하가 심함

11.1 인덱스 기본 원리

인덱스 종류

종류	설명	단점
함수기반 인덱스	- 칼럼에 함수를 적용한 값으로 만들어진 B*Tree 인덱스	- 데이터 수정, 입력 시 함수가 사용 되 부하가 발생할 수 있음
리버스 키 인덱스	- 인덱스 루트에서 리프 블록까지 어떤 값으로 탐색하더라도 읽는 블록 수가 같음을 의미	- 리버스 키 인덱스는 데이터를 거꾸로 입력하기 때문에 '=' 조건으로만 검색이 가능

11.1 인덱스 기본 원리

인덱스 종류

종류	설명	단점
클러스터 인덱스	- 클러스터 키 값이 같은 레코드가 한 블록에 모이도록 저장하는 구조	- 클러스터 인덱스를 스캔 시 Random 액세스가 값 하나당 한 번씩만 발생
클러스터형 인덱스 / IOT(Index-Organized Table)	- Index 구조로 생성한 테이블	- 데이터 입력 시 성능이 느림

11.1 인덱스 기본 원리

인덱스 튜닝 기초 – 인덱스 컬럼 가공

인덱스 칼럼을 조건절에서 가공할 경우 인덱스를 사용하지 않는다.

테스트 Script

```
CREATE INDEX IDX_EMP_01 ON EMP(EMPNO, DEPTNO);
```

```
EXPLAIN PLAN -- EXPLAININ PLAN 선언부
```

```
SET STATEMENT_ID = 'PLAN1' INTO PLAN_TABLE -- SQL에 PLAN1이라는 ID  
부여
```

```
FOR
```

```
SELECT * FROM EMP WHERE TO_NUMBER(EMPNO) = 7782;
```

```
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY -- PLAN_TABLE에 저장된 실행계획  
조회  
( 'PLAN_TABLE', 'PLAN1', 'ALL' ));
```

11.1 인덱스 기본 원리

인덱스 튜닝 기초 - 인덱스 컬럼 가공

PLAN_TABLE_OUTPUT

Plan hash value: 3956160932

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		1	39	3 (0)	00:00:01
* 1	TABLE ACCESS FULL	EMP	1	39	3 (0)	00:00:01

위 실행계획에서 나온 것처럼 인덱스 컬럼을 가공하면 인덱스를 사용하지 않는다.

부정형 비교 (<>, is null, is not null)를 사용해도 인덱스를 사용하지 않는다.

11.1 인덱스 기본 원리

인덱스 튜닝 기초 - 목시적 형변환

인덱스 칼럼을 조건절에서 가공할 경우 인덱스를 사용하지 않는다.

테스트 Script

```
CREATE INDEX IDX_EMP_02 ON EMP(job);
```

```
EXPLAIN PLAN -- EXPLAININ PLAN 선언부
```

```
SET STATEMENT_ID = 'PLAN1' INTO PLAN_TABLE -- SQL에 PLAN1이라는 ID 부여
```

```
FOR
```

```
SELECT * FROM EMP WHERE JOB = 1231231;
```

```
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY -- PLAN_TABLE에 저장된 실행계획  
조회  
( 'PLAN_TABLE', 'PLAN1', 'ALL' ));
```

11.1 인덱스 기본 원리

인덱스 튜닝 기초 - 묵시적 형변환

PLAN_TABLE_OUTPUT

Plan hash value: 3956160932

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		1	39	3 (0)	00:00:01
* 1	TABLE ACCESS FULL	EMP	1	39	3 (0)	00:00:01

Predicate Information (identified by operation id):

1 - filter(TO NUMBER("JOB")=1231231)

문자로 입력해서 조회를 했지만 묵시적 형변환으로 숫자타입으로 변환한 뒤에 실행 된다.

이때 인덱스를 타지 않고 FULL SCAN으로 실행계획이 될 수 있다.

11.2 테이블 액세스 최소화

인덱스 ROWID에 의한 테이블 Random 액세스

SQL에서 참조하는 컬럼이 인덱스에 있지 않으면 ‘테이블 Random 액세스’가 발생

테스트 Script

```
CREATE INDEX IDX_EMP_01 ON EMP(EMPNO, DEPTNO);
```

```
EXPLAIN PLAN -- EXPLAININ PLAN 선언부
```

```
SET STATEMENT_ID = 'PLAN1' INTO PLAN_TABLE -- SQL에 PLAN1이라는 ID  
부여
```

```
FOR
```

```
SELECT * FROM EMP WHERE EMPNO = '7782';
```

```
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY -- PLAN_TABLE에 저장된 실행계획  
조회  
( 'PLAN_TABLE', 'PLAN1', 'ALL' ));
```

11.2 테이블 액세스 최소화

인덱스 ROWID에 의한 테이블 Random 액세스

PLAN_TABLE_OUTPUT

Plan hash value: 10975450

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		1	87	1 (0)	00:00:01
1	TABLE ACCESS BY INDEX ROWID BATCHED	EMP	1	87	1 (0)	00:00:01
* 2	INDEX RANGE SCAN	IDX_EMP_01	1		1 (0)	00:00:01

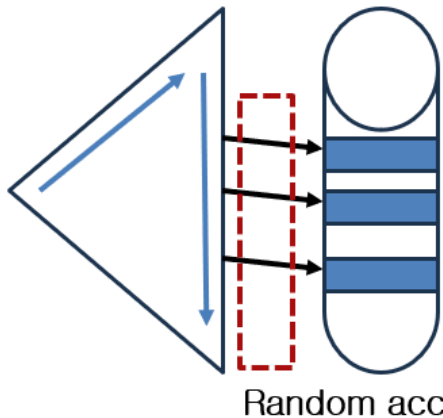
INDEX RANGE SCAN 으로 조회를 했지만 **SELECT 절에서 전체 컬럼을 조회하여 테이블 Random 액세스**가 발생

‘TABLE ACCESS BY INDEX ROWID BATCHED’ 항목이 테이블 Random 액세스가 발생된 것을 의미한다.

인덱스 스캔 후 **ROWID를 통해 데이터 블록을 읽어서 조회**

11.2 테이블 액세스 최소화

인덱스 ROWID에 의한 테이블 액세스 구조



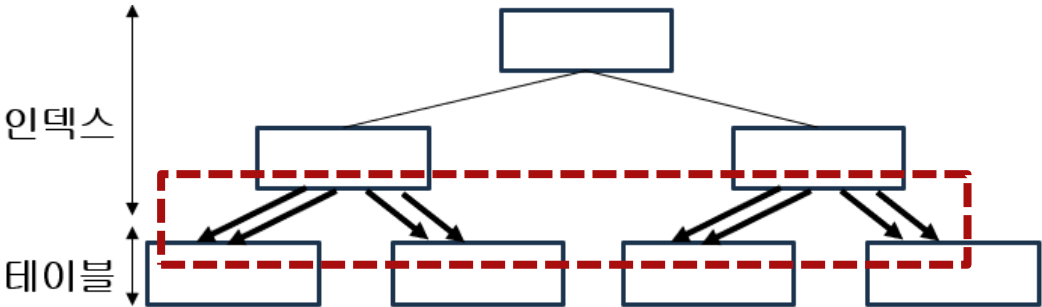
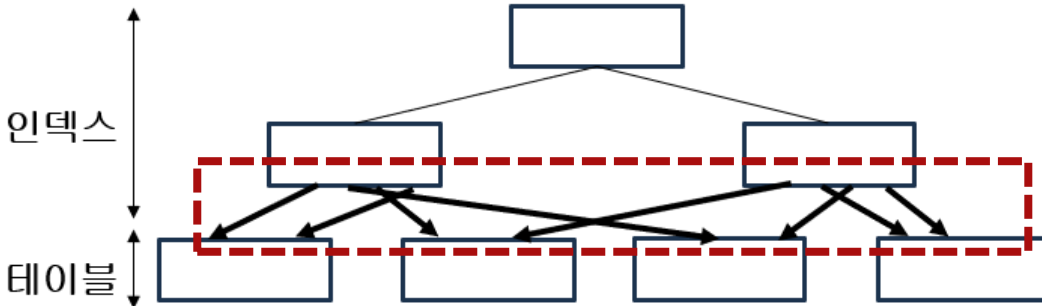
rowid는 '물리적 주소정보'로 **오브젝트 번호, 데이터 파일 번호, 블록 번호**로 구성
인덱스에서 테이블로 직접 연결되는 구조가 아니고 **해시 키 값을 통해 데이터 블록 주소(DBA, Data Block Address)**를 사용해 **데이터 블록을 읽는다.**

NO	설명
1	인덱스에서 ROWID를 읽고 DBA(Data Block Address)를 해시함수로 해시 값 확인
2	해시 값을 이용해 해시 버킷을 탐색
3	해시 버킷에 연결된 해시 체인을 스캔하여 블록 헤더를 탐색
4	해시 체인에서 블록 헤더를 찾으면 그곳에 저장된 포인터를 이용해 버퍼 블록을 읽기
5	해시 체인에서 블록 헤더를 찾지 못하면, LRU 리스트를 스캔해 Free 버퍼를 탐색 ※ 디스크에서 읽은 블록을 적재하기 위해 빈 캐시 공간을 찾는 과정
6	LRU 리스트에서 Free 버퍼를 찾지 못하면 Dirty버퍼를 디스크에 기록해 Free 버퍼확보
7	Free 버퍼를 확보하고 난 뒤 디스크에서 블록을 읽어 캐시에 적재

11.2 테이블 액세스 최소화

클러스터링 팩터(Clustering Factor)

인덱스 내의 데이터 정렬 순서와 물리적인 데이터 저장 순서 간의 관계를 나타내는 값

구분	개념도
클러스터링 팩터가 좋은 경우	 인덱스와 테이블 정렬 순서가 100% 일치
클러스터링 팩터가 좋지 않은 경우	 인덱스와 테이블 정렬 순서가 불일치

11.2 테이블 액세스 최소화

인덱스 손익분기점

인덱스 액세스가 Full Table Scan보다 느린 경우

인덱스 rowid로 테이블 액세스는 **Random 액세스**인 반면, Full Table Scan은 **Sequential 액세스** 방식

디스크 I/O 시, 인덱스 rowid의 테이블 액세스는 **Single Block Read** 방식을 사용하지만 Full Table Scan은 **Multiblock Read** 방식을 사용

인덱스 손익분기점 극복방안

NO	설명
1	테이블을 인덱스 구조로 생성 (Oracle : IOT, SQL Server : 클러스터형 인덱스), 항상 정렬된 상태 그리고 리프 블록이 곧 데이터 블록이어서 추가적인 Random 액세스가 불필요
2	SQL Server의 Include Index로 리프 노드에 인덱스 키 이외에 미리 지정된 컬럼 값 을 함께 저장
3	Oracle의 클러스터 테이블로 키 값이 같은 레코드를 같은 블록에 저장 하기 때문에 키 값 별로 Random 액세스가 1번씩만 발생
4	파티셔닝 기법으로 자주사용되는 대량범위 조건의 컬럼 기준 으로 파티셔닝 해 일부만 읽도록 처리
5	부분범위처리 원리를 이용해 데이터 일부만 읽어서 처리

11.2 테이블 액세스 최소화

테이블 액세스 최소화 튜닝

유형	설명
인덱스 컬럼 추가	- 인덱스 컬럼 이 외의 컬럼을 WHERE절에 사용하면 테이블 액세스가 발생됨 ※ 단, 신규추가보다는 기존 인덱스에 컬럼 추가를 고려하는게 좋음
Covered Index	- 모든 컬럼을 인덱스에 포함 시키는 방법 (SQL Server)
Include Index	- 인덱스 키 외에 미리 지정한 칼럼을 리프 레벨에 함께 저장 하는 기능 (SQL Server) - 컬럼을 최대 1,023 개 까지 지정 가능하며 탐색에는 사용되지 않고 필터에만 사용
IOT, 클러스터형 인덱스, 클러스터 테이블 활용	- Oracle IOT, SQL Server 클러스터형 인덱스로 테이블 Random 액세스를 최소화 - 단, 해시 클러스터를 사용 시 '=' 조건만 검색되는 컬럼을 해시 키로 사용해야 함
수동으로 클러스터링 팩터 높이기	- 테이블은 데이터가 무작위로 입력되지만 인덱스는 정해진 키 순으로 정렬되 클러스터링 팩터가 좋지 않음 - Reorg 로 해당 인덱스 기준으로 테이블을 재생성해 클러스터링 팩터를 높이는 방법 ※ 단, 인덱스가 많을 경우 기준컬럼에 따라서 클러스터링 팩터가 낮아지는 인덱스도 존재할 수 있다.
배치 I/O	- 읽는 블록마다 I/O Call을 발생시키는 비효율을 줄이기 위해 디스크 I/O Call을 일정량 모아 두었다가 한번에 처리 하는 기능 ※ 단, 인덱스 정렬 순서와 다르게 데이터가 조회 될 수 있음

11.3 인덱스 스캔 효율화

인덱스 선행 컬럼이 범위조건일 때의 비효율

인덱스 컬럼이 모두 등치(=) 조건으로 사용되면 읽은 리프블록을 테이블 액세스로 사용이 되지만 범위로 사용하게 되면 읽은 리프블록이 조건을 만족하지 않아 버려야 하는 비효율이 존재함

후행 컬럼이
범위조건 사례

사원정보 인덱스

직업	부서코드	사원번호	급여
ANALYST	20	7902	3000
CLERK	10	7934	1300
CLERK	20	7369	800
CLERK	30	7900	950
MANAGER	10	7782	2450
MANAGER	20	7566	2975
MANAGER	30	7698	2850
PRESIDENT	10	7839	5000
SALESMAN	30	7499	1600
SALESMAN	30	7521	1250
SALESMAN	30	7654	1250
SALESMAN	30	7844	1500



WHERE 직업 = 'MANAGER' AND 부서코드 BETWEEN '10' AND '20'

11.3 인덱스 스캔 효율화

인덱스 선행 컬럼이 범위조건일 때의 비효율

WHERE 직업 = 'MANAGER' AND 부서코드 BETWEEN '10' AND '20'

선행 컬럼이
범위조건 사례

사원정보 인덱스

부서코드	직업	사원번호	급여
10	CLERK	7934	1300
10	MANAGER	7782	2450
10	PRESIDENT	7839	5000
20	ANALYST	7902	3000
20	CLERK	7369	800
20	MANAGER	7566	2975
30	CLERK	7900	950
30	MANAGER	7698	2850
30	SALESMAN	7499	1600
30	SALESMAN	7521	1250
30	SALESMAN	7654	1250
30	SALESMAN	7844	1500

} 사용이 안됨
-> 비효율 존재

범위 조건이 선행 컬럼일 경우 조건에 만족하지 않는 레코드는 사용되지 않는다.

11.3 인덱스 스캔 효율화

범위조건을 In-List로 전환

WHERE 직업 = 'MANAGER' AND 부서코드 IN ('10', '20')

선행 컬럼이
범위조건 사례

사원정보 인덱스

부서코드	직업	사원번호	급여
10	CLERK	7934	1300
10	MANAGER	7782	2450
10	PRESIDENT	7839	5000
20	ANALYST	7902	3000
20	CLERK	7369	800
20	MANAGER	7566	2975
30	CLERK	7900	950
30	MANAGER	7698	2850
30	SALESMAN	7499	1600
30	SALESMAN	7521	1250
30	SALESMAN	7654	1250
30	SALESMAN	7844	1500

} 사용이 안됨
-> 비효율 존재

In-List 바꿀 경우 LIST안의 인수 개수 만큼 수직 탐색이 발생됨
※ 단, In-List로 바꿀 경우 개수가 많지 않아야 합니다.

11.3 인덱스 스캔 효율화

범위조건을 2개 이상 사용할 때의 비효율

범위 조건이 2개 이상일 경우 첫번째 조건으로 범위를 결정하고 두번째 조건은 필터 조건으로만 사용이 된다.

WHERE 부서코드 LIKE :dept || '%' AND 직업 :job || '%' :dept = '1', :job = 'M'

사원정보 인덱스

부서코드	직업	급여
10	CLERK	1300
10	MANAGER	2450
10	PRESIDENT	5000
15	ANALYST	3000
15	CLERK	800
15	MANAGER	2975
20	CLERK	950
20	MANAGER	2850
20	SALESMAN	1250
20	SALESMAN	1250
20	SALESMAN	1500
20	SALESMAN	1600

부서코드에서
조회 범위를 결정

직업은 M으로 시작하는
문자열 필터링

즉, 선행 컬럼의 범위조건을
전체 스캔해야 하는 비효율이
존재

11.3 인덱스 스캔 효율화

범위조건을 2개 이상 사용할 때의 비효율

범위 조건 2개를 사용한 목적에 따라 SQL을 변경하는 것이 좋다.

※ 부서코드가 입력되지 않을 수 있기 때문에 이 조건을 사용한 것이라고 가정을 할 경우 다음과 같이 변경하는 것이 좋다.

SQL을 분리해 작성

<SQL1>

WHERE 직업 :job || '%'

<SQL2>

WHERE 부서코드 = :dept AND 직업 :job || '%'

UNION ALL을 이용하는 방법

WHERE 부서코드 IS NULL AND 직업 :job || '%'

UNION ALL

WHERE 부서코드 IS NOT NULL AND 부서코드 = :dept AND 직업 :job || '%'

11.4 인덱스 설계

인덱스 설계 고려사항

NO	설명
1	조건절에서 항상 사용되거나 또는 자주 사용되는 컬럼
2	등치(=) 조건으로 자주 사용되는 컬럼을 선행 컬럼으로 배치
3	소트 오퍼레이션을 생략하도록 하기 위해 칼럼을 추가 ※ 인덱스는 항상 정렬 상태를 유지하므로 order by, group by와 같은 소트 연산을 생략
4	선택도(selectivity)가 충분히 낮은지가 검토 ※ 인덱스 스캔으로 테이블 액세스하는 양이 일정 수준을 넘는 순간 Table Full Scan보다 느려짐
5	쿼리 수행 빈도
6	업무상 중요도
7	클러스터링 팩터
8	데이터량
9	DML 부하(= 기존 인덱스 개수, 초당 DML 발생량, 자주 갱신되는 칼럼 포함 여부 등)
10	저장 공간
11	인덱스 관리 비용

11.4 인덱스 설계

소트 오퍼레이션이란?

정렬에 사용되는 연산

유형	설명
sort aggregate	전체 Row 대상의 집계 쿼리 사용 시 발생
sort order by	Order by 절 사용 시 발생
sort group by(hash group by)	Group By절이 있는 집계 쿼리에서 발생
sort unique(hash unique)	집합 연산(union, interest, minus), Distinct 시 발생
sort join	소트 머지 조인 시 발생되나 Outer table의 인덱스를 사용하면 발생안됨
window sort	분석함수(Window Function) 사용시 발생

선택도(Selectivity)란?

데이터 집합에서 **특정 값을 얼마나 잘 선택**할 수 있는지의 지표

$$\text{선택도} = \text{카디널리티} / \text{전체 레코드 수} * 100$$

선택도는 0~1까지 값이 나오는데 선택도가 1이라면 모든 값은 중복이 없는 고유 값임을 나타낸다.

※ 카디널리티 : 특정 데이터 집합의 유니크(Unique)한 값의 개수